

Phase 1 Bug Fixes

This document details the critical bugs found and fixed in the Phase 1 implementation of the BDI Kernel.

Bug 1 (P0 - CRITICAL): Fiber Spinning After Entry Returns

Description

When a fiber's entry function completes normally (by returning), the `fiber_entry_wrapper()` function marks the fiber as `FIBER_STATE_DEAD` but then executes an infinite loop with `while (1) { __asm__ __volatile__("pause"); }` without yielding control back to the scheduler.

Impact

- **Severity:** Critical (P0)
- Any fiber that exits without explicitly yielding will spin forever on that CPU
- The scheduler never regains control to schedule other fibers or return to the caller
- Even simple unit tests like `test_fiber_scheduler` would hang indefinitely
- Makes the fiber system completely unusable for any real workload

Root Cause

The `fiber_entry_wrapper()` function had no mechanism to return control to the scheduler after marking a fiber as DEAD. The infinite pause loop was intended to prevent undefined behavior, but it prevented the scheduler from ever running again.

The architecture uses cooperative scheduling where fibers must explicitly yield. However, there was no way for a completed fiber to yield back to the scheduler because:

1. No global or thread-local scheduler reference was available to the fiber
2. The wrapper had no way to call `fiber_scheduler_yield()`

Fix Implementation

Files Modified:

- `moduler_kernel/performance/phase1/fibers/fiber.c`
- `moduler_kernel/performance/phase1/fibers/fiber_scheduler.c`

Changes:

1. **Added thread-local scheduler reference** (`fiber.c`):

```
```c
static __thread void* g_current_scheduler = NULL;

void fiber_set_scheduler(void* scheduler) {
 g_current_scheduler = scheduler;
}
```
```

1. **Updated `fiber_entry_wrapper`** (`fiber.c`):

```
```c
```

```

static void fiber_entry_wrapper(void) {
 fiber_t* fiber = g_current_fiber;
 if (fiber && fiber->entry) {
 fiber->entry(fiber->arg);
 fiber->state = FIBER_STATE_DEAD;
 }

 // BUG FIX: Yield back to scheduler instead of spinning
 if (g_current_scheduler) {
 fiber_scheduler_yield(g_current_scheduler);
 }

 // Fallback infinite loop (should never reach here)
 while (1) {
 asm volatile("pause");
 }
}

```

## 2. Updated scheduler to set scheduler reference ( fiber\_scheduler.c ):

- Added `extern void fiber_set_scheduler(void* scheduler);` declaration
- Call `fiber_set_scheduler(scheduler)` in `fiber_scheduler_run()` and `fiber_scheduler_yield()`

## 3. Updated fiber\_scheduler\_yield to handle DEAD fibers ( fiber\_scheduler.c ):

```

c
// Only re-enqueue if fiber is not DEAD
if (current->state != FIBER_STATE_DEAD) {
 enqueue_fiber(scheduler, current);
}

```

## Validation

- Unit test `test_fiber_scheduler` now completes successfully without hanging
- Fibers that simply return from their entry function are properly cleaned up
- Scheduler correctly schedules remaining fibers after one completes
- No performance regression observed

# Bug 2 (P1 - HIGH): SPSC Ring Allocation Alignment Violation

## Description

The `spsc_ring_create()` function calls `aligned_alloc(CACHE_LINE_SIZE, total_size)` where `total_size = sizeof(spsc_ring_t) + capacity * sizeof(void*)`. For most capacity values, this size is NOT a multiple of 64 bytes (CACHE\_LINE\_SIZE).

## Impact

- **Severity:** High (P1)
- Violates C17 §7.22.3.1 requirement: "The value of alignment shall be a valid alignment supported by the implementation and the value of size shall be an integer multiple of alignment"

- Can cause `aligned_alloc()` to return NULL even when sufficient memory is available
- Invokes undefined behavior on some platforms
- Makes ring allocation unreliable and unpredictable

## Root Cause

The capacity is rounded up to the next power of two, but this does not guarantee that `sizeof(spsc_ring_t) + capacity * sizeof(void*)` is a multiple of 64 bytes.

### Example:

- `sizeof(spsc_ring_t)` might be 80 bytes (with padding)
- For capacity = 16:  $80 + 16 * 8 = 208$  bytes (not a multiple of 64)
- For capacity = 32:  $80 + 32 * 8 = 336$  bytes (not a multiple of 64)

## Fix Implementation

### File Modified:

- `moduler_kernel/performance/phase1/rings/spsc_ring.c`

### Changes:

```
// Allocate ring structure + data array
size_t total_size = sizeof(spsc_ring_t) + capacity * sizeof(void*);

// BUG FIX 2: Round total_size up to multiple of CACHE_LINE_SIZE
// C17 §7.22.3.1 requires size to be an integer multiple of alignment
total_size = ((total_size + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE) * CACHE_LINE_SIZE;

spsc_ring_t* ring = aligned_alloc(CACHE_LINE_SIZE, total_size);
```

The fix uses the standard ceiling division formula to round up to the nearest multiple of `CACHE_LINE_SIZE`.

## Validation

- Ring allocation succeeds for all capacity values
- No allocation failures observed in testing
- Memory usage slightly increased (by at most 63 bytes per ring) but acceptable
- No performance regression

## Bug 3 (P1 - HIGH): MPSC Ring Allocation Alignment Violations

### Description

The `mpsc_ring_create()` function makes TWO `aligned_alloc(CACHE_LINE_SIZE, ...)` calls, and NEITHER size is guaranteed to be a multiple of 64 bytes:

1. Ring structure: `sizeof(mpsc_ring_t) + capacity * sizeof(void*)`
2. Sequence array: `capacity * sizeof(atomic_size_t)`

### Impact

- **Severity:** High (P1)

- Same C17 §7.22.3.1 violation as Bug 2, but affects TWO allocations
- For capacities that are not multiples of eight, both allocations violate the alignment requirement
- Can cause NULL returns or undefined behavior
- Makes MPSC ring allocation even more unreliable than SPSC

## Root Cause

Same as Bug 2, but compounded by having two separate allocations:

### Example for capacity = 16:

- Ring: `sizeof(mpsc_ring_t) + 16 * 8` = likely not a multiple of 64
- Sequences: `16 * 8 = 128 bytes` (happens to be multiple of 64, but not guaranteed)

### Example for capacity = 10 (rounded to 16):

- Sequences: `16 * 8 = 128 bytes` (OK)

### Example for capacity = 5 (rounded to 8):

- Sequences: `8 * 8 = 64 bytes` (OK)

### Example for capacity = 3 (rounded to 4):

- Sequences: `4 * 8 = 32 bytes` (NOT a multiple of 64!)

## Fix Implementation

### File Modified:

- `moduler_kernel/performance/phase1/rings/mpsc_ring.c`

### Changes:

```
// Allocate ring structure + data array
size_t total_size = sizeof(mpsc_ring_t) + capacity * sizeof(void*);

// BUG FIX 3: Round total_size up to multiple of CACHE_LINE_SIZE
total_size = ((total_size + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE) * CACHE_LINE_SIZE;

mpsc_ring_t* ring = aligned_alloc(CACHE_LINE_SIZE, total_size);
if (!ring) {
 return NULL;
}

// Allocate sequence array
// BUG FIX 3: Round sequence array size up to multiple of CACHE_LINE_SIZE
size_t seq_size = capacity * sizeof(atomic_size_t);
seq_size = ((seq_size + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE) * CACHE_LINE_SIZE;

ring->sequences = aligned_alloc(CACHE_LINE_SIZE, seq_size);
```

Both allocations now properly round their sizes to multiples of `CACHE_LINE_SIZE`.

## Validation

- MPSC ring allocation succeeds for all capacity values
- Both ring and sequence array allocations comply with C17 requirements
- No allocation failures in testing
- Memory overhead acceptable (at most 63 bytes per allocation)
- No performance regression

---

## Summary

---

All three bugs have been fixed and validated:

1. **Bug 1 (P0)**: Fibers now properly yield back to scheduler when they complete
2. **Bug 2 (P1)**: SPSC ring allocations comply with C17 alignment requirements
3. **Bug 3 (P1)**: MPSC ring allocations (both ring and sequences) comply with C17 alignment requirements

## Testing Results

- All existing unit tests pass
- `test_fiber_scheduler` completes without hanging
- Ring allocation tests succeed for various capacity values
- No performance regressions observed
- No memory leaks detected

## Code Quality

- Minimal, focused changes
- Clear comments explaining each fix
- Preserved existing functionality and performance
- Followed existing code style and conventions